

On-line Packing and Covering Problems

JÁNOS CSIRIK
GERHARD J. WOEGINGER

1 Introduction

This chapter deals with a couple of problems that are all related to the classical bin packing problem. In this problem, one is given a list L of items $\langle a_1, a_2, \dots, a_n \rangle$, each item $a_i \in (0, 1]$, and the goal is to find a packing of these items into a minimum number of unit-capacity bins. This packing problem is one of the basic problems in Theoretical Computer Science and Combinatorial Optimization. It arises in many real world applications: e.g. in the (real-world) packing of (real-world) bins, in memory allocation in paged computer systems, in assigning newspaper articles to newspaper pages, in loading trucks, in packet routing in communication networks, in assigning commercials to station breaks on television, in cutting-stock problems, etc. The bin packing problem is well-known to be NP-hard [54], and thus research has concentrated on the study and development of approximation algorithms.

A packing algorithm is called *on-line* if it packs every item a_i solely on the basis of the sizes of the items a_j , $1 \leq j \leq i$, i.e., without any information on subsequent items. The decisions of the algorithm are irrevocable; packed items cannot be repacked at later times. The bin packing problem holds a special place, both in the history of approximation algorithms and in the history of on-line algorithms. Essentially, bin packing was the first problem that was carefully investigated in these two directions. Moreover, it was in the area of bin packing where for the first time the quality of an on-line algorithm was measured against the optimal off-line strategy (Johnson [71]), and it was in the area of bin packing

where for the first time an adversary argument was used for deriving lower bound results (Yao [111]).

We survey *worst case* results and *average case* results for the classical bin packing problem, for generalizations to higher dimensions, for variants with several bin types, and for bin covering problems. We mention another survey by Coffman, Garey and Johnson [23] that covers the one dimensional topics. This chapter is structured as follows: The worst case results are contained in Sections 3–7, and the average case results in Sections 8–10. First, Section 2 collects the basic definitions on worst case ratios and expected ratios of approximation algorithms. Section 3 concentrates on the classical one dimensional bin packing problem. We describe old, new and very new algorithms and discuss lower bound techniques. Section 4 deals with higher dimensional generalizations of the bin packing problems where one has to pack vectors or boxes into boxes or strips. Section 5 treats dynamic bin packing (where the items do not only arrive but also leave after some time) and variable sized bin packing (where bins of different sizes are available). Section 6 deals with the one dimensional bin covering problem: Here the goal is to pack a list of items into a maximum number of bins such that the contents of each bin is at least one. Section 7 deals with some variants of on-line packing that are only close to being on-line: here we have some limited lookahead to some of the future items or earlier packed items may be rearranged in some limited way. Section 8 deals with average case results for the classical bin packing problem, Section 9 with average case results for higher dimensional generalizations, and Section 10 closes with average case results for covering problems.

2 Basic definitions

This section summarizes the essential definition for the worst case analysis and for the average case analysis of bin packing problems. For a list $L = \langle a_1, a_2, \dots, a_n \rangle$ of items and an approximation algorithm A , we denote by $\text{OPT}(L)$ and $A(L)$, respectively, the number of bins of size 1 used by an optimum algorithm and the number of bins used by algorithm A to pack the input list L . The *absolute worst case ratio* of A , denoted by R_A , is given by

$$R_A = \sup_L \{A(L)/\text{OPT}(L)\}.$$

The *asymptotic worst case ratio* R_A^∞ of algorithm A is defined by

$$R_A^n = \max\{A(L)/\text{OPT}(L) \mid \text{OPT}(L) = n\},$$

$$R_A^\infty = \limsup_{n \rightarrow \infty} R_A^n.$$

The experiences of the last 25 years suggest that the asymptotic worst case ratio is the more reasonable measure of performance for the quality of a bin packing algorithm: It is robust against anomalies with a small number of bins in the optimum packing, and it also allows the packing algorithm more freedom while

packing the first few bins. Throughout this chapter, we will often simply write *worst case ratio* instead of ‘asymptotic worst case ratio’. In some applications, an a priori upper bound α ($0 < \alpha \leq 1$) on the size of all items is known (intuitively, it is clear that small items are easier to pack than large items). To measure the performance of algorithms on lists with small items, the *parametric worst case ratio* $R_A^\infty(\alpha)$ is introduced by

$$R_A^n(\alpha) = \max\{A(L)/\text{OPT}(L) \mid \text{OPT}(L) = n \text{ and all } a_i \in L \text{ are } \leq \alpha\},$$

$$R_A^\infty(\alpha) = \limsup_{n \rightarrow \infty} R_A^n(\alpha).$$

In the *average case analysis* of bin packing problems, a standard convention is to assume that all items in the input list L have independent, identically distributed sizes. We denote a list L with n random items by $L_n = (X_1, X_2, \dots, X_n)$, and we write $X_i \sim F$ to denote that the common distribution is F . With this convention, almost everything related to lists becomes a random variable: $\text{OPT}(L_n)$, $A(L_n)$, $R_A(L_n) = A(L_n)/\text{OPT}(L_n)$, and $s(L_n)$ (the total item size). We will also investigate $W_A(L_n) = A(L_n) - s(L_n)$ which is the empty (= wasted) space when packing list L_n according to algorithm A . For measuring the quality of the average case behaviour of A , we define

$$R_A^n(F) = E[R_A(L_n)] = E \left[\frac{A(L_n)}{\text{OPT}(L_n)} \right]$$

and

$$W_A^n(F) = E[A(L_n) - s(L_n)]$$

for lists L_n with item size distribution F . The *asymptotic expected ratio* of A under F is given by

$$R_A^\infty(F) = \lim_{n \rightarrow \infty} R_A^n(F).$$

The *uniform distribution* on $[a, b]$ with $0 \leq a < b \leq 1$ will be denoted by $U(a, b)$. In this chapter, we will mainly deal with average case results for such uniform distributions.

Next, let $t_1 = 2$ and $t_{i+1} = t_i(t_i - 1) + 1$ for $i \geq 1$. This sequence starts with $\langle 2, 3, 7, 43, 1807, 3263443, \dots \rangle$ and its growth is doubly exponential. The sequence turned out to be essential in the analysis of on-line bin packing algorithms, and also in proving lower bound results for on-line bin packing. Moreover, the closely related number

$$h_\infty = \sum_{i=1}^{\infty} \frac{1}{t_i - 1} = 1 + \frac{1}{2} + \frac{1}{6} + \frac{1}{42} + \frac{1}{1806} + \dots \approx 1.69103$$

will appear many times in our statements.

3 Worst case analysis for one dimensional bin packing

This section is divided into four subsections: Subsection 3.1 deals with the ‘classical’ packing algorithms like NEXT FIT, FIRST FIT, BEST FIT, and WORST FIT. These are the algorithms that were introduced and analyzed in the beginning of the 1970s. Subsection 3.2 considers *bounded-space* algorithms. Such algorithms manage to pack with only a small number of simultaneously open bins. Subsection 3.3 describes the algorithms that currently yield the best worst case performance of all known on-line algorithms. Finally, Subsection 3.4 investigates lower bounds on the asymptotic worst case ratio of one dimensional bin packing algorithms.

3.1 Classical algorithms

The simplest, oldest and fastest on-line algorithm for one dimensional bin packing is the NEXT FIT (NF): NF always packs a newly arriving item a_i into a unique, so-called *active* bin. In case item a_i does not fit into the active bin, the current active bin is closed (and never used again) and an empty bin is opened and becomes the next active bin. Clearly, the running time of NF is linear in the number of items packed. It is easy to show that $NF(L) \leq 2OPT(L)$ for all lists L . Moreover, the list $L = \langle \frac{1}{2}, \varepsilon, \frac{1}{2}, \varepsilon, \frac{1}{2}, \varepsilon, \dots \rangle$ consisting of n pairs $\frac{1}{2}, \varepsilon$ where $0 < \varepsilon < \frac{1}{n}$ demonstrates that this bound is tight. Hence, $R_{NF}^\infty = 2$ holds.

Theorem 1. (Johnson [71, 72]) *For $1/2 \leq \alpha \leq 1$, $R_{NF}^\infty(\alpha) = 2$ and for $\alpha < 1/2$, $R_{NF}^\infty(\alpha) = 1/(1 - \alpha)$.*

The worst case example above illustrates that NEXT FIT sometimes wastes a lot of space, since it does not reuse the empty space in earlier packed bins. This naturally leads to the FIRST FIT (FF) algorithm: Here, during the packing we never close active bins. When packing some new item a_i , FF puts it into the lowest indexed bin into which it will fit. A new bin is started only in the case where a_i will not fit into any non-empty bin. The $O(n \log n)$ running time of FF is slower than the $O(n)$ running time of NEXT FIT. However, the worst case ratio of FF indeed yields an improvement over NEXT FIT.

Theorem 2. (Johnson, Demers, Ullman, Garey, Graham [73]) *Let $m \in \mathbb{N}$ with $\frac{1}{m+1} < \alpha \leq \frac{1}{m}$. Then for the FIRST FIT algorithm,*

- $R_{FF}^\infty(\alpha) = 17/10$ holds for $m = 1$, and
- $R_{FF}^\infty(\alpha) = (m + 1)/m$ for $m \geq 2$.

The proof of the 17/10-result is based on a so-called *weighting function* $w : [0, 1] \rightarrow [0, 8/5]$ which assigns to the size a_i of every item a corresponding weight $w(a_i)$. Intuitively, this weight $w(a_i)$ measures the space that item a_i uses when packed according to FF (where space equals the size of item plus some wasted space). For $w(L) = \sum_{i=1}^n w(a_i)$, it then can be shown that (i) $FF(L) \leq w(L) + 2$ and that (ii) $w(L) \leq (17/10)OPT(L)$ holds.

Algorithm	Time	R_A^∞	$R_A^\infty(\frac{1}{2})$	$R_A^\infty(\frac{1}{3})$	$R_A^\infty(\frac{1}{4})$
NEXT FIT	$\Theta(n)$	2.0	2.0	1.5	1.33...
WORST FIT	$\Theta(n \log n)$	2.0	2.0	1.5	1.33...
ALMOST WORST FIT	$\Theta(n \log n)$	1.7	1.5	1.33...	1.25
FIRST FIT	$\Theta(n \log n)$	1.7	1.5	1.33...	1.25
BEST FIT	$\Theta(n \log n)$	1.7	1.5	1.33...	1.25

Table 1. Asymptotic worst case ratios for the classical algorithms.

At first sight, the packing rule of FF (‘use the lowest indexed bin’) seems to be rather primitive. Obvious variants for this packing rule yield the BEST FIT (BF), WORST FIT (WF) and ALMOST WORST FIT (AWF): BEST FIT behaves like FIRST FIT, except that the new item a_i is placed in the bin into which it will fit with the smallest gap left over (ties are broken arbitrarily). WORST FIT puts a_i into the non-empty bin with the largest gap (ties are broken arbitrarily), starting a new bin only if this largest gap is not big enough. ALMOST WORST FIT tries to put a_i into the non-empty bin with second largest gap first, and in case a_i does not fit AWF behaves like WORST FIT.

All these three algorithms belong to the class of so-called ANY FIT algorithms: An ANY FIT (AF) algorithm *never* puts an item a_i into an empty bin unless the item does not FIT into ANY partially filled bin. Similarly, an ALMOST ANY FIT (AAF) algorithm is an ANY FIT algorithm that never puts an item into a partially filled bin with lowest level, unless there is more than one bin with lowest level or unless that bin is the only one that has enough room. The following surprising result indicates a borderline for all algorithms defined till now.

Theorem 3. (Johnson [72]) *For any $0 < \alpha \leq 1$,*

- *For every AF algorithm A , $R_A^\infty(\alpha) \geq R_{FF}^\infty(\alpha)$ holds.*
- *For every AAF algorithm A , $R_A^\infty(\alpha) = R_{FF}^\infty(\alpha)$ holds.*

Table 1 states the worst case ratios for the introduced algorithms together with some of their parametric worst case ratios. Note that ALMOST WORST FIT – somewhat surprisingly – performs much better than WORST FIT does.

3.2 Bounded-space algorithms

An on-line bin packing algorithm is said to use *k*-bounded-space if for each item a_i , the choice of bins into which it may be packed is restricted to a set of k or fewer *active* bins, where each bin becomes active when it receives its first item; once a bin is declared *inactive* (or *closed*), it can never become active again. Since for $k = 1$, the only reasonable *k*-bounded-space algorithm is the NEXT FIT with worst case ratio 2, we assume from now on that $k \geq 2$.

The bounded-space restriction models situations in which bins are exported once they are packed. Consider the problem of packing trucks at a loading dock that has positions for only k trucks. If the next item to be packed does not fit into any of the trucks currently backed up to the dock, we will need a new truck. If there are already k trucks present, one of them will have to drive away to make room (and presumably to start making its deliveries). Alternatively, consider a communication channel in which information moves in large fixed-size blocks. If these blocks are filled with smaller packets of various sizes that must be assigned to blocks as they arrive at the entrance to the channel, we have an on-line bin packing problem. If the buffer for the channel input has only room for k blocks, we also are subject to the k -bounded memory restriction.

The most natural bounded-space bin packing algorithms are defined via simple *packing* rules for items and *closing* rules for bins: A new item may always be packed into the lowest indexed bin (as in the FIRST FIT) or into the bin with the smallest remaining gap (as in the BEST FIT). If the new item does not fit into any active bin, some active bin has to be closed; in this case one may always choose the lowest indexed bin (the FIRST bin) or the fullest bin (the BEST bin). This leads to four algorithms AFF_k , AFB_k , ABB_k , and ABF_k that are classified according to these rules: Here A stands for *Algorithm*, the second letter denotes the packing rule (Best fit or First fit), the third letter denotes the closing rule (Best bin or First bin), and k is an upper bound on the number of active bins.

- Algorithm AFF_k . This algorithm was already introduced in 1974 by Johnson [72] under the name NEXT- k FIT (NF_k). Just fifteen years later, Csirik and Imreh [37] discovered a sequence of worst case examples that imply $R_{NF_k}^\infty \geq 1.7 + \frac{3}{10(k-1)}$. Mao [90] proved a matching upper bound on $R_{NF_k}^\infty$.
- Algorithm AFB_k . Zhang [112] showed that for this algorithm, the analysis of Mao [90] can be adapted to prove the same worst case bound as for AFF_k . Thus, $R_{AFB_k}^\infty = 1.7 + \frac{3}{10(k-1)}$.
- Algorithm ABF_k . Mao [89] proved that $R_{ABF_k}^\infty = 1.7 + \frac{3}{10k}$.
- Algorithm ABB_k . This algorithm was investigated by Csirik and Johnson [38] under the name k -BOUNDED BEST FIT (BBF_k). In some sense, BBF_k is the most clever of the four algorithms since it uses the better packing and the better closing rule. Csirik and Johnson confirmed in a rather sophisticated proof that best is indeed better than first: $R_{BBF_k}^\infty = 17/10$ always holds, independently of the value of k .

Summarizing, the worst case guarantees of all four algorithms tend to 1.7 as the number k of active bins tends to infinity. The big surprise is that BBF_2 performs as well as FIRST FIT and BEST FIT, while using only *two* active bins. However, since all four algorithms fulfill a kind of ANY FIT constraint, they can not beat FIRST FIT. To find an algorithm with $R_A^\infty < 1.7$, it thus is necessary to start new bins even if the current item fits into one of the active bins.

In this spirit, C.C. Lee and D.T. Lee [83] introduced the HARMONIC algorithm H_k . This algorithm is based on a nonuniform partition of the interval

$(0, 1]$ into k subintervals where the partitioning points are $1/2, 1/3, \dots, 1/k$. To each of these subintervals there always corresponds a single active bin and only items belonging to this subinterval are packed into this corresponding bin. If a new item arrives that does not fit into its corresponding bin, this bin is closed and a new bin is used.

Theorem 4. (Lee and Lee [83]) *As k tends to infinity, $R_{H_k}^\infty$ tends to the number $h_\infty \approx 1.69103$.*

It can be shown that algorithm H_7 (with at most seven active bins) has a worst case ratio of 1.6944 and thus beats FIRST FIT. In fact, there is another bounded-space algorithm SH_6 that beats FIRST FIT while using only *six* active bins: It behaves very similar to H_7 , but uses only a single active bin for the items in $(1/6, 1/4]$. This algorithm belongs to the class of SIMPLIFIED HARMONIC (SH_k) algorithms introduced by Woeginger [109]. The SH_k are modifications of the H_k that use a more complicated interval structure.

k	NF_k	ABF_k	BBF_k	H_k	SH_k	Champion
2	2.00000	1.85000	1.70000	2.00000	2.00000	BBF
3	1.85000	1.80000	1.70000	1.75000	1.75000	BBF
4	1.80000	1.77500	1.70000	1.71429	1.72222	BBF
5	1.77500	1.76000	1.70000	1.70000	1.70000	BBF, H , SH
6	1.76000	1.75000	1.70000	1.70000	1.69444	SH
7	1.75000	1.74286	1.70000	1.69444	1.69388	SH
8	1.74286	1.73750	1.70000	1.69388	1.69106	SH
9	1.73750	1.73333	1.70000	1.69345	1.69104	SH
∞	1.70000	1.70000	1.70000	1.69103	1.69103	H , SH

Table 2. Asymptotic worst case ratios for bounded-space algorithms, rounded to five decimal places.

A summary of the worst case ratios of the bounded-space algorithms for some small values of k is given in Table 2. In the column for H_k , the values for $k \leq 7$ are the exact worst case ratios and the values for $k \in \{8, 9\}$ are upper bounds on the worst case ratio. The tight results for $k \in \{4, 5\}$ are due to van Vliet [108], the other results are from Lee and Lee [83]. Note that the worst case ratios of all five algorithms always remain above h_∞ . In fact, for bounded-space algorithms it is not possible to do better:

Theorem 5. (Lee and Lee [83]) *Every bounded-space on-line bin packing algorithm A fulfills $R_A^\infty \geq h_\infty$.*

The worst case ratios of H_k and SH_k approach h_∞ in the limit. However, none of the known bounded-space algorithms reaches the worst case ratio h_∞ while

using a *finite* number of active bins. Deciding whether this is possible at all remains an open problem. The parametric worst case ratio of H_k was analyzed by Galambos [46] (see Table 3 for some of these results).

3.3 The current champion

The first on-line algorithm for bin packing with $R_A^\infty < h_\infty$ was the REVISED FIRST FIT (RFF) designed by Yao [111] in 1980. Both, the definition and the analysis of RFF are rather involved: RFF is essentially based on FIRST FIT, but similarly as the H_k it uses separate bins for items from the intervals $(0, 1/3]$, $(1/3, 2/5]$, $(2/5, 1/2]$, and $(1/2, 1]$, respectively. Moreover, every sixth item from the interval $(1/3, 2/5]$ receives a special treatment. Yao proved that RFF has $R_{RFF}^\infty = 5/3$.

In the same paper in which they introduced the harmonic algorithms, Lee and Lee [83] described the REFINED HARMONIC (RH) algorithm with worst case ratio $R_{RH}^\infty = 373/228 \approx 1.639$. RH is based on H_{20} , but the intervals $(1/3, 1/2]$ and $(1/2, 1]$ both are broken into two corresponding subintervals. As a next step, Ramanan, Brown, Lee and Lee [94] investigated another modification of the harmonic algorithm which they called MODIFIED HARMONIC (MH). MH is a refinement of the REFINED HARMONIC, uses a more complicated interval structure, and fulfills $R_{MH}^\infty \approx 1.616$. Ramanan et al [94] also prove that their underlying basic approach can never yield a ratio $R_A^\infty < 1.583$.

The current champion algorithm is the HARMONIC+1 of Richey [101] introduced in 1991. The definition of HARMONIC+1 uses more than 70 intervals. Its design and analysis was performed with the help of several large linear programs. Its worst case ratio is less than 1.5888.

3.4 Lower bounds

Theorem 5 yields a lower bound on the performance of bounded-space bin packing algorithms. To derive lower bounds on the performance of *general* on-line bin packing algorithms, one uses ideas from everyday life: Assume that you have to pack your suitcase. Then you will usually start with first packing the large items and afterwards you will try to fit in the smaller items. Therefore, if one wants to make some algorithm A to behave poorly, it is natural to confront it first with a set of small items. In case A packs these small items very tight, it will not be able to pack the larger items that may arrive later. In case A leaves lots of room for large items while packing the small items, the large items will not arrive. In either case, the produced packing will be very bad.

Yao [111] was the first to translate this idea into a more mathematical formulation: He first confronted the algorithm with a list consisting of n items of size $1/7 + \varepsilon$ (small items), then with n items of size $1/3 + \varepsilon$ (medium size items), and finally with n items of size $1/2 + \varepsilon$ (large items). Yao showed that for this list, every on-line algorithm must be worse than the optimum by a factor of $3/2$, either after the first or after the second or after the third sublist. Hence, every on-line bin packing algorithm fulfills $R_A^\infty \geq 1.5$. Just a short time later, Brown

[15] and Liang [87] simultaneously and independently from each other generalized this lower bound to 1.53635. They used lists with items of sizes $1/1807 + \varepsilon$, $1/43 + \varepsilon$, $1/7 + \varepsilon$, $1/3 + \varepsilon$ and $1/2 + \varepsilon$, respectively. Note that these item sizes correspond to the numbers t_i , $1 \leq i \leq 5$, introduced in Section 2 for the definition of h_∞ . The analysis in [87] is based on some hand-woven inequalities and takes some effort to read. It took more than ten years until van Vliet [106, 107] was able to give a tight analysis of the Brown/Liang construction. He did this by designing a very elegant linear programming formulation.

Theorem 6. (Van Vliet [106, 107]) *For any on-line bin packing algorithm A , the bound $R_A^\infty \geq 1.5401$ holds.*

Note that the gap between this lower bound and the upper bound of 1.5888 described in Section 3.3 is less than 0.05. Galambos [45] and Galambos and Frenk [48] simplified and extended the Brown/Liang construction to the parametric case. Combining these results with the linear programming formulation of van Vliet [106] yields the lower bounds given in Table 3. For comparison, corresponding upper bounds for some algorithms are also given.

α	1	1/2	1/3	1/4	1/5
Lower bound on $R_A^\infty(\alpha)$	1.540	1.389	1.291	1.229	1.188
Current champion	1.588	1.423	1.302	1.234	1.191
$R_{H_k}^\infty(\alpha)$	1.691	1.423	1.302	1.234	1.191
$R_{FF}^\infty(\alpha)$	1.700	1.500	1.333	1.250	1.200

Table 3. Lower bounds for the parametric case.

Chandra [18] argued that all these lower bounds can be carried over to *randomized* on-line bin packing algorithms (algorithms that are allowed to flip coins and act according to the outcome of these flips) even in the case of oblivious adversaries.

4 Worst case analysis for higher dimensional packing

The one dimensional bin packing problem has been generalized to higher dimensions in several ways. First, there is the generalization to *vectors* as treated in Subsection 4.1: Instead of packing numbers, one has to pack vectors subject to the constraint that in every bin the sum of all vectors is at most one in every coordinate. Subsection 4.2 deals with the generalization to *boxes*: Here one has to pack geometric boxes into unit cubes. The boxes must not be rotated and different boxes must not overlap. Subsection 4.3 deals with another geometric generalization called *strip packing*: The items are two dimensional objects that have to be packed into a vertical strip so as to minimize the total height of the strip needed.

4.1 Vector packing

Instead of each item being a single number, in the d -dimensional vector packing problem each item is a d -dimensional vector $a_i = (v_1(a_i), \dots, v_d(a_i))$, where $0 \leq v_j(a_i) \leq 1$ holds for $1 \leq j \leq d$. The goal is to pack all items into the minimum number of bins in such a way that in every bin the sum of all vectors is at most one in every coordinate. The vector packing problem arises as a crucial subproblem in scheduling with resource constraints (cf. Garey, Graham, Johnson and Yao [53]).

Kou and Markowsky [79] call an approximation algorithm A for d -dimensional vector packing *reasonable*, if A never yields packings in which the contents of two non-empty bins could be combined into a single bin (this obviously corresponds to the ANY FIT algorithms described in Section 3.1 for the one dimensional problem). Kou and Markowsky show that any reasonable vector packing algorithm A obeys the bound $R_A^\infty \leq d + 1$. We note that the obvious generalization of FIRST FIT to d -dimensional vector packing is reasonable and hence obeys the above-mentioned (not very impressive) bound. In 1976, Garey, Graham, Johnson and Yao [53] performed an exact analysis of the d -dimensional FIRST FIT and derived a slight improvement over $d + 1$:

Theorem 7. (Garey, Graham, Johnson, Yao [53]) *The generalization of FIRST FIT to d -dimensional vector packing has worst case ratio $R_{FF}^\infty = d + 7/10$.*

Note that this theorem reduces to the familiar 17/10 result in the one dimensional case for the standard FIRST FIT (cf. Theorem 2). Until today, these ancient results provide the best known worst case bounds for d -dimensional on-line vector packing. Especially, the question whether $R_A^\infty < d$ is possible for some on-line approximation algorithm A remains open (in fact, it even remains open for *off-line* approximation algorithms!).

Now let us turn to lower bounds on the worst case ratios of on-line algorithms for vector packing. Yao [111] proved that in a reasonable model of computation, any off-line approximation algorithms with $o(n \log n)$ running time must have a worst case ratio $R_A^\infty \geq d$. Since on-line algorithms form a subclass of the off-line algorithms, this result indicates that no fast and simple on-line vector packing algorithm can break the barrier of d . Next, we observe that the 1.54 lower bound for one dimensional bin packing (cf. Theorem 6) trivially carries over to vector packing in $d \geq 2$ dimensions: We just replace every item a_i in the one dimensional construction by a vector $(a_i, a_i, \dots, a_i)$. Slight improvements on this observation were made by Galambos, Kellerer and Woeginger [49] and then in turn were slightly improved by Blitz, van Vliet and Woeginger [14]. The exact values of these lower bounds for dimensions $2 \leq d \leq 8$ are depicted in Table 4. The underlying technique for proving all these lower bounds is essentially the same as for the one dimensional case: One designs an appropriate list L of items (L usually consists of a small number of homogeneous sublists), and then derives a number of combinatorial properties for the packing patterns of prefixes of L . The goal is to show that *every* algorithm has to perform badly on one of these prefixes. Hence, these lower bounds are tedious to derive, but nevertheless are

d	2	3	4	5	6	7	8	∞
Lower bound [49]	1.6707	1.7509	1.8003	1.8334	1.8572	1.8750	1.8889	2.000
Lower bound [14]	1.6712	1.7584	1.8326	1.8729	1.8980	1.9149	1.9271	2.000

Table 4. Lower bounds for d -dimensional on-line vector packing, rounded to four decimal places.

not impressive: as the dimension d tends to infinity, the bounds tend to 2. The main open problem in on-line d -dimensional vector packing consists in narrowing the wide gap between 2 and $d + 7/10$.

4.2 Box packing

Box packing is a purely geometric way of generalizing the one dimensional bin packing problem to d dimensions: Similarly as in the vector packing problem, each item a_i is described by a d -dimensional vector $a_i = (v_1(a_i), \dots, v_d(a_i))$, with $0 \leq v_j(a_i) \leq 1$ for $1 \leq j \leq d$. However in this version of the problem these vectors determine the dimensions of a d -dimensional box, a d -dimensional solid geometric object. The goal is to pack all items into the minimum number of bins (which now are d -dimensional unit cubes) in such a way that (i) each item is entirely contained inside its bin with all sides parallel to the sides of the bin, (ii) no two items in a bin overlap and (iii) the orientation of any item is the same as the orientation of the bin (in other words, the items must not be rotated).

Most of the approximation algorithms developed for d -dimensional box packing are off-line algorithms (e.g. Chung, Garey, and Johnson [20]). The first on-line algorithm for 2-dimensional box packing is due to Coppersmith and Raghavan [29]. The main idea of this algorithm is to first round up the larger dimension of a newly arrived item to some number in the set $\{2^{-k}, 2^{-k}/3 | k \geq 0\}$ and then to pack it according to a classification scheme, in a similar way as the HARMONIC algorithm. Since the rounding just wastes a constant factor and since the packing just wastes another constant factor, it can be shown that this yields an on-line approximation algorithm CRA with worst case ratio $R_{CRA}^\infty \leq 3.25$. Csirik, Frenk and Labbé [34] took a closer look at the packing scheme of the CRA algorithm. They showed that if one packs items belonging to the same class according to the NEXT FIT rule, then $R_{CRA}^\infty = 3.25$. However, in case one packs these items according to the FIRST FIT rule then the worst case ratio drops to $R_{CRA}^\infty \leq 3.06$. As a next result, Csirik and van Vliet [41] combined a rounding procedure that is more complex than that of CRA with a clever generalization of the harmonic algorithm to d dimensions. The resulting algorithm is the current champion for all dimensions $d \geq 2$.

Theorem 8. (Csirik and van Vliet [41]) *For d -dimensional on-line box packing, there exists an on-line algorithm with worst ratio bounded by $h_\infty^d \approx 1.69^d$.*

The situation for lower bounds for the d -dimensional box packing problem is similarly disappointing as it is in the case of d -dimensional vector packing: The first improvement over the trivial 1.54 bound induced by the one dimensional case was done by Galambos [47]. He provides a lower bound of 1.6 for the 2-dimensional box packing problem. Galambos and van Vliet [50] apply a refinement of the same proof technique to derive a better bound of 1.808. In his Ph.D. thesis [107], van Vliet extends his construction to establish a lower bound of 1.851 at the cost of having to work with *very* complex packing patterns. Currently, the best lower bound known for 2-dimensional box packing is 1.907 which is due to Blitz, van Vliet and Woeginger [14]. For 3-dimensional box packing, [14] provides a lower bound of 2.111. Any straightforward generalization of these lower bound methods to $d \geq 4$ dimensions will yield values that are smaller than 3. Since the current champions for d -dimensional box packing have worst case ratios that are exponential in d , there is ample space for improvements.

Lassak and Zhang [82] and Januszewski and Lassak [69] investigate a somewhat different type of on-line box packing. Their goal is to pack a set of boxes into a *single* unit bin. Among other results, they prove that in two dimensions every set of boxes with total area at most 0.0903 can be packed on-line into the unit square. Moreover, every set of squares with total area at most 0.3125 can be on-line packed into the unit square. For $d \in \{3, 4\}$, they show that every set of cubes with total volume at most $\frac{3}{2}(\frac{1}{2})^d$ can be packed into the d -dimensional unit cube. For $d \geq 5$, the volume bound is $2(\frac{1}{2})^d$. The result for $d \geq 5$ is as good as the best possible corresponding off-line result of Meir and Moser [91]: Two cubes of edge length $\frac{1}{2} + \epsilon$ cannot be packed together.

4.3 Strip packing and TETRIS

In $d = 2$ dimensions, there is another reasonable version of higher dimensional bin packing besides box packing. Again, all items are boxes with given heights and widths. The goal is to pack them into a vertical strip of width W so as to minimize the total height of the strip needed. Without loss of generality, we may assume that $W = 1$ and that the widths of all items are at most one (this can be reached by scaling all these numbers by an appropriate factor). The boxes must be packed orthogonally; in general, rotations are forbidden. This 2-dimensional version of bin packing is called *strip packing*.

This problem has many applications in VLSI-design or stock-cutting. The application to stock-cutting is as follows. In a variety of industrial settings, the raw material comes in rolls (rolls of cloth, rolls of paper, rolls of sheet material). From these rolls we want to cut patterns (for clothes, labels, boxes) or merely smaller and shorter rolls. In the simplest case we can view the objects we wish to cut from the rolls as being or approximating boxes. We minimize our wastage if we minimize the amount of the roll (i.e., the strip length) used. Some form of orthogonality may be justified, since in many applications the cutting is done by blades parallel or perpendicular to the length of the material, and the material may have a bias that dictates the orientation of the boxes.

Similarly as for d -dimensional box packing, most of the algorithms developed for strip packing are off-line algorithms. Typically, they preorder the items by decreasing height or by decreasing width (see e.g. the papers by Sleator [105], Baker, Coffman and Rivest [10], and Baker, Brown and Katseff [7]). The only on-line algorithms analyzed from a worst case point of view for this problem are the *shelf algorithms* introduced by Baker and Schwarz [11]. Before describing these approximation algorithms, we stress the fact that they assume that the heights of all boxes are also bounded by 1.

Note that in case all the boxes have the same height, the 2-dimensional problem collapses to the classical one dimensional problem: in an optimal packing, the bins may be placed in rows or ‘shelves’. Each shelf in the packing then corresponds to a bin and the height of the strip packing corresponds to the number of bins used. The basic idea of the shelf algorithms is to classify the arriving boxes according to their heights. The packing is then constructed as a sequence of shelves, and boxes of similar heights are packed into the same shelves. The main differences between distinct shelf algorithms consist in the way how boxes within the same class are packed. More precisely, each shelf algorithm takes a parameter $0 < r \leq 1$, which is a measure of how much wasted space can occur. All shelves have heights r^j , $j \geq 0$. Whenever a box (h_i, w_i) is to be packed, one first determines the value of j for which $r^{j+1} < h_i \leq r^j$ holds. If there is already a shelf of height r^j in the packing, the NEXT FIT SHELF algorithm checks whether the box fits into the current active one and in case it fits, it packs the box there. Otherwise, the box is placed into a newly opened shelf of height r^j which becomes the currently active one for that height. The FIRST FIT SHELF algorithm behaves analogously, but tries to fit the new box into the *first* shelf of corresponding height into which it will fit.

Theorem 9. (Baker and Schwarz [11]) *For the shelf packing algorithms NEXT FIT SHELF and FIRST FIT SHELF with parameter $0 < r \leq 1$, $R_{NFS}^\infty = 2/r$ and $R_{FFS}^\infty = 1.7/r$ holds. For $r = 0.622$, the absolute worst case ratio of FFS obeys $R_{FFS} \leq 6.9863$.*

Note that these worst case ratios approach the corresponding worst case ratios of NEXT FIT and of FIRST FIT as r approaches 1. In an analogous way, Csirik and Woeginger [42] transform the HARMONIC algorithm into the corresponding HARMONIC SHELF algorithm. The worst case ratios of the HARMONIC SHELF algorithm can be made arbitrarily close to h_∞ . Improving on h_∞ in on-line strip packing is probably difficult: Csirik and Woeginger [42] argue that no shelf algorithm can yield a worst case ratio better than h_∞ . Hence, a completely different approach will be necessary.

Lower bounds on the worst case ratio of any on-line strip packing algorithm are investigated by Baker, Brown and Katseff [8]. Among many other results for algorithms with preorderings, they prove that the *absolute* worst case ratio of any on-line strip packing algorithm A must fulfill $R_A \geq 2$. For the *asymptotic* worst case ratio, the 1.54 lower bound can be carried over from the one dimensional case to strip packing.

Li and Cheng [86] investigate the 3-dimensional version of strip packing: Here the strip is a column with a unit square cross section and the items are 3-dimensional boxes. [86] proposes and analyzes nine on-line generalizations of the 2-dimensional shelf algorithms. Each of these algorithms uses a separate packing heuristic in each dimension. The overall worst case ratio of such an algorithm equals the product of the worst case ratios of the individual heuristics used independently in different dimensions. By choosing certain parameters appropriately, one arrives at an algorithm with worst case ratio 2.897.

Up to now we assumed that every newly arrived box can be placed without problems into any free space of the size of the rectangle. Azar and Epstein [5] consider a variant that also takes into consideration the way how the box is moved to its place. In their model, a box arrives from the top of the strip and has to be moved continuously around only in the free space until it reaches its place, and afterwards cannot be moved again. As in classical strip packing, the goal is to minimize the total height of the strip needed. For the case where rotations are not allowed, [5] prove that no on-line algorithm can have a finite worst case guarantee. The argument uses boxes that become arbitrarily thin. However, if every item has a width of at least β then there exists an on-line packing algorithm with worst case ratio $O(\log(1/\beta))$, and no worst case ratio better than $\Omega(\sqrt{\log(1/\beta)})$ is possible. For the case where rotations by ninety degrees are allowed, Azar and Epstein construct an on-line algorithm with worst case ratio 4.

We close this section with a short discussion of the computer game TETRIS that also is a kind of on-line strip packing problem. Designed by the Soviet mathematician Alexey Pazhitnov in the late eighties, TETRIS won a record number of software awards in 1989. In this game, the board is ten units wide and twenty units tall; when the game starts the board is empty. Then tetrominoes (= groups of four connected cells, each cell covering exactly one grid square) appear at the top of the board and fall down. When a tetromino reaches a point where it can fall no further without two cells overlapping, it remains in that spot and the next tetromino appears at the top of the board. The player uses rotations and horizontal translations to guide the falling items. When a row is completely filled, the cells in that row are removed from the board. As soon as the packing height exceeds 20, the player loses. In his Master's thesis, Brzustowski [16] developed an adversary strategy that makes every on-line algorithm eventually lose the game. Burgiel [17] showed that even *off-line* algorithms (that know the whole sequence of tetrominoes in advance) can be forced to lose eventually: Just confront them with a sequence that alternates between left-handed and right-handed Z-shaped items. After at most 70.000 items, the player will lose.

5 Dynamic bin packing and variable sized bin packing

In **dynamic bin packing**, the items a_i only stay in the system for some (initially unknown) time period. Hence, there are two possible types of information given to the packer in every step: Either, the packer learns about the arrival of some

new item a_i (*insert event*), or the packer is told that now some earlier arrived item leaves the system (*delete event*). At any moment in time, the total size of items currently assigned to any bin must not exceed 1. This problem models memory allocation problems in paged computer systems (cf. Coffman [21]).

For an input list L and an approximation algorithm A , we denote by $A(L)$ the maximum number of bins occupied by A while it is packing L . We shall compare the quality of an approximation algorithm against an optimum packer that is allowed to repack the current packing into an optimum packing each time a new item arrives; $\text{OPT}(L)$ is the maximum number of bins used by such an optimum packer while packing L . In analogy to the asymptotic worst case ratio for classical bin packing, the worst case ratio R_A^{dyn} of algorithm A is defined by

$$R_A^{\text{dyn}} = \lim_{k \rightarrow \infty} \sup_L \{A(L)/\text{OPT}(L) \mid \text{OPT}(L) = k\}.$$

Coffman, Garey and Johnson [22] investigate a variant of FIRST FIT for dynamic bin packing: Every time a new item arrives, it is packed into the lowest indexed occupied bin that has sufficient available space; if no such bin exists, a new bin is opened. Every time an item a_i is deleted, the available space in the corresponding bin is simply increased by a_i . Coffman et al [22] prove that $2.75 \leq R_{\text{FF}}^{\text{dyn}} \leq 2.897$. A slightly modified version FFx of FF that packs every item $> 1/2$ into its own bin and never adds small items to such a large item behaves better: $2.75 \leq R_{\text{FFx}}^{\text{dyn}} \leq 2.778$ holds. Moreover, no on-line algorithm for dynamic bin packing can beat a lower bound of 2.5. The situation drastically improves in the parametric version where all item sizes are bounded by $\alpha \leq 1/2$: In these cases, worst case ratios of at most 1.788 are possible. Moreover, the lower bounds drop to $1 + \alpha + \alpha^2 + O(\alpha^3)$, and the parametric worst case ratios of the FIRST FIT match these lower bounds up to a term of order $O(\alpha^3)$.

In **variable sized bin packing**, we are given several different types $B_1, \dots, B_r$ of bins with sizes $1 = s(B_1) > s(B_2) > \dots > s(B_r)$. There is an infinite supply of bins of each size. The problem is to pack a list of items $a_i \in [0, 1]$ into a set of bins with *smallest total size* (observe that for the case where all bins are of size one, this is just the classical one dimensional bin packing problem). For a list L of items and an approximation algorithm A , let $s(A, L)$ denote the total size of bins used by algorithm A . Let $s(\text{OPT}, L)$ denote the total size of bins used in an optimal packing. Then, we measure the quality of algorithm A by

$$R_A^{\text{var}} = \lim_{k \rightarrow \infty} \sup_L \{s(A, L)/s(\text{OPT}, L) \mid \text{OPT}(L) \geq k\}.$$

In the on-line version of variable sized bin packing, every time a new bin is opened, the packing algorithm chooses which bin size to use next.

Friesen and Langston [44] give three approximation algorithms with finite worst case ratios for variable sized bin packing; only one of these three algorithms is on-line. This algorithm always chooses the largest available bin size when a new bin is opened, and otherwise behaves just like NEXT FIT. It is not surprising that it has a worst case ratio of 2, the same as NEXT FIT. In fact, *any* on-line algorithm that always chooses the *largest* available bin size has a worst

case ratio of at least 2: Consider an instance with two bin sizes $s(B_1) = 1$ and $s(B_2) = 1/2 + \varepsilon$, and an input list L consisting only of pieces of size $1/2 + \varepsilon$. A similar example demonstrates that on-line algorithms that always choose the *smallest* bin size have worst case ratios of at least 2. Therefore, Kinnersley and Langston [76] design a hybrid strategy that they call FFf . Here the ‘FF’ stands for FIRST FIT, since FFf uses the packing strategy of FIRST FIT. Moreover, FFf is based on a so-called *filling factor* f , $1/2 \leq f \leq 1$. Suppose that FFf must start a new bin as item a_i arrives. If a_i is a small item with $a_i \leq 1/2$, then FFf starts a new bin of size 1. If a_i is a large item with $a_i > 1/2$, then FFf chooses the smallest bin size in the interval $[a_i, a_i/f]$; in case no such bin size exists, FFf chooses size 1. Intuitively speaking, when FFf packs large items into new bins then it tries to make them full to at least a fraction of f . The worst case ratio of FFf is at most $1.5 + f/2$, and there exist values of f (e.g. $f = 3/5$) for which this bound is tight. Zhang [113] proves that the worst case ratio of $FF\frac{1}{2}$ equals $17/10$, thus matching the guarantee of FIRST FIT. Observe that for large items, $FF\frac{1}{2}$ always selects the smallest bin size into which they will fit.

Csirik [31] designs an on-line algorithm for variable sized bin packing that is based on the HARMONIC algorithm (cf. Section 3.2). For every item size a_i , this algorithm computes the most appropriate bin type B_j (i.e., the bin size that in some sense minimizes the expected wasted space for item size a_i). All items that are assigned to the same bin type B_j are then packed with a HARMONIC algorithm into bins of size $s(B_j)$. For any collection of bin types, the worst case guarantee of this algorithm can be made arbitrarily close to $h_\infty \approx 1.691$. Quite surprisingly, the algorithm sometimes performs even better for special collections of bin types:

Corollary 10. (Csirik [31]) *For variable sized one dimensional bin packing with two bin sizes $s(B_1) = 1$ and $s(B_2) = 0.7$, there exists an on-line approximation algorithm A with worst case ratio $R_A^{var} = 1.4$.*

Note that the bound 1.4 is smaller than the 1.5401 lower bound for classical one dimensional bin packing! A partial explanation for this phenomenon goes as follows. Although variable sized bin packing is a much harder problem than classical bin packing, it also gives the on-line packer more freedom for making decisions. Suppose that there were bin types of all sizes between $1/2$ and 1. Then the packing of large items with sizes $> 1/2$ can be done without wasting any space, and the small items with sizes $\leq 1/2$ are anyway easy to pack without wasting lots of space.

There remain many open problems in this area. Does there exist an on-line algorithm that has worst case ratio strictly less than h_∞ for all collections of bin types? Which combination of two bin types allows the smallest possible worst case ratio? What can be said about lower bounds depending on the bin sizes for variable sized bin packing?

6 Worst case analysis for bin covering problems

In the *packing* problems treated till now, the goal is to partition a set of items (numbers, vectors, boxes) into the *minimum* number of subsets such that in every subset the total size of the items does not exceed some *upper* bound. In a *covering* problem, the goal is to partition a set of items into the *maximum* number of subsets such that in every subset the total size of the items does not remain under some *lower* bound. Covering problems model a variety of situations encountered in business and in industry, from packing peach slices into tin cans so that each tin can contains at least its advertised net weight, to such complex problems as breaking up monopolies into smaller companies, each of which is large enough to be viable. Since covering problems may be considered to be a kind of inverse or dual version of the packing problem, they are sometimes also called ‘dual packing’ problems in the literature. This section deals with the one dimensional bin covering problem and with some generalizations to covering by vectors, cubes and boxes.

In the one dimensional bin covering problem, the goal is to pack a list $L = \langle a_1, a_2, \dots, a_n \rangle$ into a maximum number of bins of size 1 such that the contents of each bin is at least one. Similarly to the definitions introduced in Section 2, we will denote by T_A^∞ the asymptotic worst case ratio of an approximation algorithm for bin covering ($A(L)$ and $\text{OPT}(L)$ denote the number of bins in the packing constructed by algorithm A and in the optimal packing, respectively). It is defined by

$$T_A^n = \min\{A(L)/\text{OPT}(L) \mid \text{OPT}(L) = n\},$$

$$T_A^\infty = \liminf_{n \rightarrow \infty} T_A^n.$$

Since we deal with a maximization problem, we have that the larger the worst case ratio of an algorithm, the better the approximation algorithm. The bin covering problem was for the first time investigated in the Ph.D. thesis [3] of Assmann and in the journal article [4] by Assmann, Johnson, Kleitman and Leung. They considered the following adaptation of NEXT FIT, called DUAL NEXT FIT (DNF): DNF always keeps a single active bin. Newly arriving items a_i are packed into the active bin until the active bin is full (i.e., has contents at least one). Then the active bin is closed and another (empty) bin becomes the active bin. It is not too difficult to see that DNF fulfills $T_{DNF}^\infty = \frac{1}{2}$. However, here analogous modifications of FIRST FIT, BEST FIT and HARMONIC *do not* improve this worst case ratio $\frac{1}{2}$ (e.g. modifying FIRST FIT is useless, since after filling a bin, placing further items into it does not make sense). In fact, from the worst case point of view, algorithm DNF is the best possible on-line bin covering algorithm:

Theorem 11. (Csirik, Totik [40]) *Every on-line bin covering algorithm A fulfills $T_A^\infty \leq \frac{1}{2}$.*

Woeginger and Zhang [110] consider bin covering with variable sized bins: There are several different types $B_1, \dots, B_r$ of bins with sizes $1 = s(B_1) > s(B_2) > \dots > s(B_r)$; there is an infinite supply of bins of each size. The problem is to cover with a given list of items $a_i \in [0, 1]$ a set of bins with *largest total size*. [110] determine for every finite collection of bin sizes the worst case ratio of the best possible on-line covering algorithm. This worst case ratio mainly depends on the largest gap between consecutive bin sizes of the bins with $s(B_j) > 1/2$.

Alon, Azar, Csirik, Epstein, Sevastianov, Vestjens and Woeginger [2] investigate the vector covering problem: Here the input consists of a list L of d -dimensional vectors in $[0, 1]^d$, and the goal is to assign L to a maximum number of bins, subject to the constraint that in every bin the sum of all vectors is at least one in every coordinate. For every $d \geq 2$, Alon et al [2] demonstrate the existence of an on-line vector covering algorithm with ratio $1/(2d)$: The main idea is to partition the list of vectors into d sublists in such a way that the i -th sublist ($1 \leq i \leq d$) contains at least a fraction of roughly $1/d$ of the total size of all the i -th coordinates in L . Somewhat surprisingly, this can be done in an on-line fashion. At the same time, the vectors that are assigned to the i -th sublist are used to cover the i -th coordinate of unit bins by applying DUAL NEXT FIT. Moreover, Alon et al [2] show that for $d \geq 2$ every on-line algorithm must fulfill $T_A^\infty \leq 2/(2d + 1)$.

There is only a small number of results known for geometric covering by boxes and cubes. In this version of the problem, the boxes (respectively cubes) are allowed to overlap; rotations of the items are *not* allowed. All known results concern on-line covering of a *single* d -dimensional unit cube. Kuperberg [80] developed a beautiful on-line algorithm for covering the unit cube by smaller cubes; this algorithm packs the cubes along a space-filling Peano curve and uses the current position on this curve as a measure of progress. Kuperberg proves that in d -dimensional space, every sequence of cubes whose sum of volumes is greater than 4^d admits an on-line covering of the unit cube. Kuperberg's paper triggered a race which led to more and more elaborate variants. The results of Januszewski and Lassak [66, 68], of Lassak [81] and of Januszewski, Lassak, Rote and Woeginger [70] finally led to the following theorem.

Theorem 12. (Januszewski, Lassak, Rote and Woeginger [70]) *For $d \geq 2$, every sequence of cubes in d -dimensional space whose sum of volumes is greater than $2^d + 3$ admits an on-line covering of the unit cube.*

It is interesting to observe that the estimate in Theorem 12 is almost as good as the best possible off-line estimate: For every $d \geq 2$, the sequence consisting of $2^d - 1$ cubes with edge length $1 - \varepsilon$ can not be used to cover the unit cube (since every corner of the unit cube has to be covered by its own small cube). Consequently, the best off-line estimate must be at least $2^d - 1$. In 1982 Groemer [55] showed that every sequence of cubes in d -dimensional space whose sum of volumes is greater than $2^d - 1$ indeed allows an *off-line* covering of the d -dimensional unit cube. For $d = 2$, Theorem 12 gives an (on-line) upper bound of 7 on the volume. This bound can be improved down to 5.265 (cf. Januszewski and Lassak [68]).

There are also several results on covering the unit cube by axes-parallel boxes (whose edge lengths are bounded by one) or by other convex bodies with bounded diameters (cf. Januszewski and Lassak [65, 67]). We only mention that every sequence of axes-parallel boxes with edge lengths bounded by 1 and total volume at least $2 \cdot 8^{d-1} - 2^{d-1}$ allows an on-line covering of the d -dimensional unit cube.

7 Semi on-line bin packing

This section deals with several variants of bin packing that are ‘almost’ on-line: Either the input list is presorted, or the packer has some lookahead to some of the future items, or earlier packed items may be rearranged in some limited way.

First, let us assume that the input list is *presorted*. In case the list is in increasing order, this is of no great help to us: the lower bound techniques in Section 3.4 all work with such increasing lists, and hence the 1.5401 lower bound in Theorem 6 applies to this case, too. However, in case the items are sorted by decreasing size, the problem becomes much easier: The NEXT FIT algorithm on a decreasing list is called the NEXT FIT DECREASING (NFD) algorithm, and the FIRST FIT algorithm on a decreasing list is called the FIRST FIT DECREASING (FFD) algorithm. The worst case ratio of NFD equals 1.691 (Baker and Coffman [9]) and the worst case ratio of FFD equals 11/9 (cf. Johnson [72], Baker [6], Csirik [32]). This is quite an improvement over the worst case bounds 2 and 1.7, respectively, for arbitrary lists. On the other hand, Csirik, Galambos and Turán [36] showed that every on-line algorithm for decreasing lists must have a worst case ratio of at least $8/7$. Their argument uses two lists L_1 and L_2 where L_1 contains n items of size $1/3 + 2\varepsilon$ and L_2 contains $2n$ items of size $1/3 - \varepsilon$, where $0 < \varepsilon < 1/12$. It can be seen that any on-line algorithm must be at least a factor of $8/7$ away from the optimum either if it has to pack list L_1 , or if it has to pack the concatenated list L_1L_2 .

Next, let us reconsider the k -bounded-space restriction introduced in Section 3.2. According to Theorem 5, each bounded-space on-line algorithm has worst case ratio at least h_∞ . Moreover, it is an open problem whether there actually exists a bounded-space algorithm that reaches the worst case ratio h_∞ while using a finite number of active bins. Grove [56] considers a variant of bounded-space bin packing with *bounded lookahead*. Bounded lookahead means that for some fixed constant W , an item a_i need not be packed until one has looked through all the items a_j , $j > i$, for which $\sum_{l=i}^j a_l \leq W$ holds. For every variant with k -bounded space and lookahead W , the h_∞ lower bound from Theorem 5 still applies. However, if k and W are fixed at sufficiently large values, then it can be shown that the worst case ratio of the REVISED WAREHOUSE (RW) algorithm introduced by Grove reaches this lower bound. Algorithm RW is based on a weighting function $w : [0, 1] \rightarrow [0, 3/2]$. For each list L of items, the total weight of the items in this list fulfills $w(L) \leq h_\infty \text{OPT}(L)$. If the lookahead W is sufficiently large, then the average weight of the bins closed by RW is at least 1. Hence, $\text{RW}(L) \leq w(L) + k$ holds; this implies $R_{RW}^\infty = h_\infty$.

Galambos and Woeginger [51] introduced yet another variant of bounded-space bin packing where repacking of the currently active bins is allowed. Here repacking means that the packer is allowed to take all items out of the currently active bins and to reassign them before packing the next item. Also for this variant the h_∞ lower bound in Theorem 5 still applies. Similarly as Grove [56], Galambos and Woeginger [51] use a weighting function $w' : [0, 1] \rightarrow [0, 3/2]$ such that for each list L , $w'(L) \leq h_\infty \text{OPT}(L)$ holds. Moreover, function w' has the following property: If one takes three partially filled bins B_1 , B_2 and B_3 and repacks their items according to FFD, then afterwards either one bin has become empty or in one bin the total w' -weight of the items is at least 1. The algorithm REP_3 introduced in [51] uses three active bins; every time a new item arrives, it repacks the active bins by FFD, then closes all bins with total w' -weight at least 1, and finally puts the new item into an empty bin. Since the w' -weight of every closed bin is at least 1, $\text{REP}_3(L) \leq w'(L) + 3$ and $R_{\text{REP}_3}^\infty = h_\infty$.

Next, let us turn to a variant where the repacking is less restricted (observe that if we allowed completely unlimited repacking then we would end up with an off-line problem). Gambosi, Postiglione and Talamo [52] allow for every new item a small number of *simple* repacking moves. In a simple repacking move, one takes a single item or a bundle of items that currently are assigned to the same bin and transports this item or bundle to another bin. Gambosi et al [52] suggest two algorithms for this model. Their first algorithm uses at most three simple repacking moves per new item, works with a simple classification of the items into four types, and has a worst case ratio $R_A^\infty = 3/2$. Their second algorithm uses up to seven simple repacking moves per new item, works with a more complicated item classification with six types, and has a worst case ratio $R_A^\infty = 4/3$. Hence, both algorithms beat the 1.5401 lower bound for the classical model! Ivković and Lloyd [62, 64] give an even more sophisticated algorithm that achieves a worst case ratio of $5/4$ with $O(\log n)$ simple repacking moves per new item and five types of items. Their algorithm is designed to handle the more general dynamic bin packing problem (cf. Section 5). Moving bundles of items is an essential component of the algorithms in [52, 62, 64]. Let us define a *very simple* repacking move to be a simple move that only transports a single item. It can be shown [63] that any algorithm that only makes a bounded number of very simple repacking moves per new item has a worst case ratio of at least $4/3$.

8 Average case analysis for one dimensional bin packing

Shortly after the systematic worst case analysis of bin packing algorithms had started, also the first papers on average case analysis of bin packing appeared. Since then a lot of work has been done in this field: several excellent review papers (Coffman, Lueker and Rinnooy Kan [26], and Coffman and Shor [27]) were written, and there are also two books (Hofri [60], and Coffman and Lueker [25]) that partly deal with the probabilistic analysis of bin packing. It turned out that the average case behaviour gives an important supplement to the worst case analysis of approximation algorithms.

This section is divided into two subsections: Subsection 8.1 deals with the average case behaviour of NEXT FIT, HARMONIC, and of variants of these algorithms on uniform distributions. It will turn out that all these algorithms have their asymptotic expected ratios bounded away from 1. Subsection 8.2 treats FIRST FIT and BEST FIT that are asymptotically optimal. Moreover, the expected wasted space for on-line algorithms on uniform distributions is studied in some detail.

8.1 Next Fit, Smart Next Fit, and Harmonic

Let us start with discussing the NEXT FIT algorithm. As a first step, Shapiro [102] performed an analysis of NF for exponential distributions. Then, based on a Markov model approach, Coffman, So, Hofri, and Yao [28] analyzed the behaviour of NF for $U(0, 1)$ (recall that $U(0, 1)$ is the continuous uniform distribution on $[0, 1]$). Their analysis concentrates on the behaviour of the level of the currently last closed bin. They prove that the distribution of this level converges to the distribution x^3 , $0 \leq x \leq 1$. Since this distribution has mean $3/4$, we get the following result.

Theorem 13. (Coffman, So, Hofri, Yao [28]) $R_{NF}^\infty(U(0, 1)) = 4/3$.

Hofri [59] presented an analysis of NF for arbitrary distributions. Such general questions are quite hard to handle, and so he derived sets of equations that implicitly describe the expected number of bins that NF uses for packing n items. For the uniform distribution $U(0, 1)$, however, it is easy to read off from his results that

$$E[NF(L_n)] = \begin{cases} 1, & n = 1 \\ \frac{2}{3}n + \frac{1}{6}, & n \geq 1. \end{cases}$$

Since for $U(0, 1)$, $E[OPT(L_n)]$ grows like $n/2$, this confirms the result stated in Theorem 13.

In order to analyze NF for $U(0, a)$, Karmarkar [74] formulated NF as a Markov process with discrete time steps and continuous state space. He overcomes technical difficulties by a novel technique of converting the differential equation governing the corresponding steady-state distribution into a matrix-differential equation, which then can be solved by standard methods. For $1/2 < a \leq 1$, he gives the closed-form result

$$\rho = \frac{1}{12a^3}(15a^3 - 9a^2 + 3a - 1) + \frac{(1-a)^2}{2\sqrt{2}a} + \tanh \frac{1-a}{\sqrt{2}a}$$

where $E[NF(L_n)] \sim \rho n$ as $n \rightarrow \infty$. These results agree exactly with the computational experiments of Ong, Magazine, and Wee [92]. In particular, Ong et al [92] had observed a strong drop in the efficiency of NF for values of a between 0.8 and 0.9. Karmarkar's work shows that $R_{NF}^\infty(U(0, a))$ indeed takes a local maximum at $a = 0.841$.

Ramanan [93] devised a modification of NF that is called SMART NEXT FIT (SNF) and differs from NF in only one respect: In case the new item does

not fit into the active bin, the item is put into a newly opened bin, and then the *fuller one* of these two bins is closed and the other one becomes the active bin. Hence, SNF uses 2-bounded space. By applying the method of Karmarkar [74], Ramanan analyzes SNF and derives the asymptotic expected ratios of SNF for $U(0, a)$ with $1/2 \leq a \leq 1$. In particular, $R_{SNF}^\infty(U(0, 1)) = 1.227$ holds. Hence, SNF is indeed an improvement over the 1.333 bound of NF.

The HARMONIC algorithm is easy to analyze, even for arbitrary distributions. Essentially, the analysis reduces to counting or estimating the expected number of items in every harmonic interval $(1/(j+1), 1/j]$. For the distribution $U(0, 1)$ the following holds: if the number k of active bins goes to infinity, then the asymptotic expected ratio of H_k tends to $\pi^2/3 - 2 \approx 1.289$. But even for $k = 3$ active bins, the asymptotic expected ratio of H_3 is about 1.306. These results were derived by many authors independently from each other: e.g. by Csirik and Máté [39], by Hofri and Kahmi [61], or by Lee and Lee [84]. Ramanan and Tsuga [95] investigated the asymptotic expected ratio of the MODIFIED HARMONIC algorithm. They showed that for $U(0, 1)$, $R_{MH}^\infty(U(0, 1)) \leq 1.276$ holds, which is slightly better than the bound for HARMONIC. Moreover, they propose and analyze a simple modification MH^* of the MH that brings the asymptotic expected ratio down to 1.189.

Summarizing the results on asymptotic expected ratios for $U(0, 1)$ described till now, we get that

$$\left(\begin{array}{c} MH^* \\ 1.189 \end{array} \right) < \left(\begin{array}{c} SNF \\ 1.227 \end{array} \right) < \left(\begin{array}{c} MH \\ 1.276 \end{array} \right) < \left(\begin{array}{c} H_{1000} \\ 1.289 \end{array} \right) < \left(\begin{array}{c} H_3 \\ 1.306 \end{array} \right) < \left(\begin{array}{c} NF \\ 1.333 \end{array} \right)$$

holds. It is also interesting to note that according to Coffman and Shor [27], each on-line algorithm A that gets along with k -bounded-space must satisfy the inequality

$$W_A^n(U(0, 1)) \geq \frac{n}{16(k+1)}.$$

Hence, on average every bounded-space algorithm A wastes a constant fraction of $s(L_n)$, and thus it can not be asymptotically optimal. For $k = 2$, the above expression yields that no on-line algorithm using 2-bounded-space can have an asymptotic expected ratio better than $25/24 \approx 1.041$. SMART NEXT FIT reaches 1.227 with 2-bounded-space. Based on extensive computational experiments, Csirik and Johnson [38] conjecture that their 2-bounded-space BBF_2 algorithm (cf. Section 3.2) has $R_{BBF_2}^\infty(U(0, 1)) = 1.178$. Hence, there remains quite a gap to be closed between 1.041 and 1.178.

8.2 First Fit, Best Fit, and the problem of wasted space

Although FIRST FIT and BEST FIT are outperformed by the bounded-space algorithms from the worst case point of view, they behave much better than their competitors from the average case of view:

Theorem 14. (Bentley, Johnson, Leighton, McGeoch, McGeoch [13])
 $R_{FF}^\infty(U(0, 1)) = 1$.

Also for BEST FIT, $R_{BF}^\infty(U(0, 1)) = 1$ holds true. Now immediately the question arises, *how fast* these ratios do tend to 1. This question can be answered via investigating the wasted space. For the FIRST FIT, already Bentley et al [13] showed that $W_{FF}^n(U(0, 1))$ grows with $O(n^{\frac{4}{3}})$. Later, Shor [103] gave tighter bounds and proved that the expected wasted space of FF is $\Omega(n^{\frac{2}{3}})$ and $O(n^{\frac{2}{3}}\sqrt{\log n})$. Finally, Coffman, Johnson, Shor and Weber [24] settled the problem completely by showing that $W_{FF}^n(U(0, 1))$ is $\Theta(n^{\frac{2}{3}})$, i.e., that Shor's lower bound gives the right order of magnitude. For the BEST FIT algorithm, Shor [103] proved that the expected wasted space $W_{BF}^n(U(0, 1))$ is $\Omega(\sqrt{n} \log^{\frac{3}{4}} n)$, and Leighton and Shor [85] proved that it is $O(\sqrt{n} \log^{\frac{3}{4}} n)$. The proofs of most of these results rely on results for certain planar matching problems derived by Ajtai, Komlós and Tusnády [1]. The connection between the bin packing problem and the matching problem was drawn by Karp, Luby and Marchetti-Spaccamela [75]. Similar results were achieved by Rhee and Talagrand [99], [100] for more general distributions.

Summarizing, FF does not waste too much space, and BF wastes even less. What can be said in general on the wasted space of approximation algorithms? Since $W_{OPT}^n(U(0, 1)) = \Omega(\sqrt{n})$, even an off-line algorithm cannot do better than $\Omega(\sqrt{n})$. In 1986, Shor [103] gave a very interesting partial answer to this question for on-line algorithms: Consider an on-line algorithm A that receives k items with sizes uniformly distributed according to $U(0, 1)$. The value of k is chosen with equal probability from the integers between 1 and n , and A is given no information about the value of k until it has received and packed the last item. Call such an on-line algorithm A an *open* on-line algorithm. Shor proved that each open on-line algorithm must have an expected waste of $\Omega(\sqrt{n \log n})$. Algorithm BF is just a factor of $O(\log^{\frac{1}{4}} n)$ away from this lower bound. In 1991, Shor [104] detected an open on-line algorithm with expected waste $O(\sqrt{n \log n})$, thus matching his lower bound. The algorithm roughly proceeds as follows.

Let us assume for the moment that we a priori know the number k of items that will be packed. Let $p = \sqrt{k/\log k}$. For $1 \leq i \leq p$, let S_i be the interval $[\frac{i-1}{2p}, \frac{i}{2p}]$ and let L_i be the interval $[1 - \frac{i}{2p}, 1 - \frac{i-1}{2p}]$. Hence, the union of the intervals L_i covers the large items $> 1/2$ and the union of the intervals S_i covers the small items $\leq 1/2$. The bins are divided into $p + 3$ classes labeled $B_0, B_1, \dots, B_{p+2}$. Into any bin, the algorithm will put at most one large and at most one small item. Large items from interval L_i only go into bins of class B_{i-1} or B_i , and small items from interval S_i only go into bins of class B_{i+1} or B_{i+2} . This ensures that an item from interval L_i will get paired with an item from interval S_{i-1} or smaller, so that the sizes of the two items always will sum to at most 1. The exact assignment of items to bin classes can be modeled via a *balls in buckets game* for which Shor [104] designs a very sophisticated algorithm. In the end, the overall wasted space is half the expected number of bins with one item in them (since the expected size of an item is $1/2$ and since we never put more than two items into a bin), which can be shown to be $O(k/p + p \log p) = O(\sqrt{k \log k})$. To turn this procedure into an open on-line algorithm, one dynamically changes p every time the number of items received so far doubles. Even if one starts over with a new set of bins each time p is updated and leaves the previously half-filled

bins forever half-empty, one still gets $O(\sqrt{n \log n})$ expected wasted space.

If an on-line algorithm A is told the number of items in advance, it is easy to reach the best possible bound $W_A^n(U(0, 1)) = \Omega(\sqrt{n})$: It simply packs the first $\lfloor k/2 \rfloor$ items into separate bins, and then packs the remaining $\lceil k/2 \rceil$ items by applying BEST FIT (Shor [103]). Also in the case where the total number of items is unknown, but the items are presorted into decreasing order, the best possible bound $\Omega(\sqrt{n})$ can be reached: Based on preliminary work by Frederickson [43], Knödel [77] and Lueker [88] proved independently of each other that in this case the FIRST FIT DECREASING algorithm yields wasted space $\Theta(\sqrt{n})$. These questions were investigated for general distributions by Rhee and Talagrand [96, 97]. A variety of results that yield asymptotically optimal algorithms for various special cases are contained in the papers of Hoffman [57], Knödel [78], Csirik [30], Csirik and Galambos [35], and Chang, Wang, and Kankanhalli [19].

9 Average case analysis for higher dimensional packing

In Section 4, we surveyed worst case results for three types of higher dimensional bin packing problems: For vector packing, for box packing, and for strip packing. For average case analysis, Karp, Luby and Marchetti-Spaccalema [75] investigated the average-case behaviour of the optimal packing for the vector packing and box packing under uniform distribution on $(0, 1)^d$. They also gave asymptotically optimal off-line algorithms for these problems. We are not aware of any results for the *on-line vector packing* problem. However, it seems plausible that the d -dimensional variant of FIRST FIT introduced by Garey, Graham, Johnson, and Yao [53] (cf. Theorem 7) is asymptotically optimal if the vectors are independent, identically and uniformly distributed in $[0, 1]^d$. For *box packing*, Csirik and van Vliet [41] introduced an on-line algorithm that may be considered to form a higher dimensional generalization of the HARMONIC algorithm (cf. Theorem 8). Indeed, it can be shown that the asymptotic expected ratio of this algorithm equals $(\pi^2/3 - 2)^d$ in d dimensions, if the vectors that describe the side lengths of the boxes are distributed independently, identically and uniformly in $[0, 1]^d$. For $d = 1$, this takes us back to the asymptotic expected ratio of the HARMONIC algorithm for one dimensional bin packing (cf. Section 8.1). Chang, Wang and Kankanhalli [19] defined an algorithm HashPacking for (both open and closed) d -dimensional box packing which achieves average wasted space of order $O(n^{(d+1)/(d+2)})$.

In the remainder of this section we will deal with the average case analysis of strip packing (cf. Section 4.3). As before, the strip has width 1. Here we assume that the boxes are given in the form (h_i, w_i) where h_i and w_i are independent variables. Hofri [58] investigates a primitive shelf algorithm in which only one shelf is packed at a time. The boxes are always packed by NEXT FIT onto this single active shelf. The height of the active shelf is not determined until some item fails to fit, whereupon it is fixed at the maximum item height in the shelf and a new shelf is opened above the filled shelf. It is easy to see that the worst

case ratio of this algorithm is unbounded: Just consider the list that consists of n sublists with items $(\frac{1}{n}, 1)$ and $(1 - \frac{1}{n}, \frac{1}{n})$. Then the optimum strip packing has height 2, whereas the algorithm constructs a packing of height n . However, the algorithm has a reasonable average case behaviour: If $h_i \sim U(0, 1)$ and $w_i \sim U(0, 1)$, then the algorithm constructs packings with an expected height $E[H] \sim 0.572n$ and an expected wasted area of $\sim 0.337n$.

Bartholdi, vande Vate and Zhang [12] analyzed a generalization of the NEXT FIT SHELF algorithm described in Section 4.3. In their analysis, they allow arbitrary finite sequences $1 = s_0 > \dots > s_k > s_{k+1} = 0$ of feasible shelf heights. Remember that for every height s_i , NFS keeps an active shelf into which all boxes with height in $(s_{j+1}, s_j]$ are packed. Whenever a new box does not fit into its shelf, this shelf is closed and a new shelf of height s_j is opened. For the boxes (h_i, w_i) , it is assumed that the widths w_i are independent, identically distributed according to $U(0, 1)$ and that the heights h_i are independent, identically distributed according to a cumulative distribution F on $[0, 1]$. F is assumed to have density function f and a mean μ . Bartholdi et al [12] proved that under these assumptions, the expected height of an NFS packing for n boxes equals

$$E[H] = \frac{2n}{3} \sum_{j=0}^k s_j p_j + \frac{1}{6} \sum_{j=0}^k s_j (1 + np_j(1 - p_j)^{n-1} - (1 - p_j)^n),$$

where for $0 \leq j \leq k$, $p_j = F(s_j) - F(s_{j+1})$ denotes the probability that the height of an item lies in the interval $(s_{j+1}, s_j]$, so that it is packed onto a shelf of height s_j . From this result one easily derives results on the expected wasted area (which equals $E[H] - \mu/2$):

- For the original NFS as introduced by Baker and Schwarz [11] with shelf heights $s_j = r^j$ and $h_i \sim F = U(0, 1)$, the expected wasted area equals ωn with $\omega = \frac{2}{3}(1 + r^{2k+1})/(1 + r) - \frac{1}{4}$.
- For $h_i \sim F = U(0, 1)$ and shelf heights $s_j = (k - j)/k$, the expected wasted area equals ωn with $\omega = \frac{1}{3}(1 + 1/k) - \frac{1}{4}$.

Moreover, for any given distribution F the expected wasted area is minimized for the shelf heights defined by $s_{k+1} = 0$ and $s_j - s_{j+1} = (F(s_{j+1}) - F(s_{j+2}))/f(s_{j+1})$ for $0 \leq j \leq k$. All these strip packing algorithms have expected waste linear in the number of items. Shor [104] points out that his average case results for the one dimensional case also yield an on-line strip packing algorithm with expected waste $\Theta(\sqrt{n \log n})$ and that this result is best possible.

10 Average case analysis for bin covering

In this section, we discuss some average case results for bin covering problems. Let us start with the one dimensional case. Already in 1983, Assmann [3] proved that for a list L_n with n items $\sim U(0, 1)$, $E[\text{OPT}(L_n)] = n/2 - \Omega(n^{1/2})$ holds for n large enough. Several years later, Csirik, Frenk, Galambos and Rinnooy Kan [33] showed that the $\Omega(n^{1/2})$ in this formula is in fact $\Theta(n^{1/2})$. Hence,

the behaviour of $E[\text{OPT}(L_n)]$ is completely understood for the uniform distribution. Rhee and Talagrand [98] investigated the optimal solution for general distribution too.

By modeling the DUAL NEXT FIT algorithm (cf. Section 6) as a Markov process with discrete time steps (= the arrival of an item) and continuous state space (= the level of the current bin), Assmann [3] showed that $E[\text{DNF}(L_n)]$ grows like $n/e \approx n/2.718$ for the distribution $U(0, 1)$. Hence, the asymptotic expected ratio of DNF for $U(0, 1)$ equals $2/e \approx 0.735$. It is perhaps surprising that presorting the elements in decreasing order does not help for this problem: In this case the asymptotic expected ratio of DNF becomes worse and drops to $4 - \pi^2/3 \approx 0.710$ (see Csirik et al [33]). By using strong techniques from renewal theory, a general result for distributions $U(0, a)$ can be derived.

Theorem 15. (Csirik, Frenk, Galambos, Rinnooy Kan [33]) *If the items in L_n are distributed according to $U(0, a)$, $0 \leq a \leq 1$, then $E[\text{DNF}(L_n)]$ grows like n/τ where*

$$\tau = \sum_{\ell=0}^{\lfloor 1/a \rfloor} (-1)^\ell \frac{1}{\ell!} \left(\frac{1}{a} - \ell \right)^\ell \exp \left(\frac{1}{a} - \ell \right).$$

As a last result for the one dimensional problem we mention that Csirik and Totik [40] designed an asymptotically optimal on-line algorithm for $U(0, 1)$.

The only average case results known for higher dimensional covering are for 2-dimensional vector covering. If the vectors are uniformly and independently distributed on $[0, 1]^2$, then $\lim_{n \rightarrow \infty} E[\text{OPT}(L_n)]/n = 1/2$. The obvious extension DNF_2 of the DUAL NEXT FIT algorithm to 2-dimensional vector covering was analyzed by Csirik et al [33]. They proved that the asymptotic expected ratio of DNF_2 equals $2/\tau \approx 0.633$, where $\tau = 2e - \sum_{t=0}^{\infty} 1/(t!)^2 \approx 3.156$.

Acknowledgements

This work was partially supported by the Start-program Y43-MAT of the Austrian Ministry of Science and by the Project 20u2 of the Austro-Hungarian Action Fund.

References

1. M. Ajtai, J. Komlós, and G. Tusnády. On optimal matching. *Combinatorica*, 4:259–264, 1984.
2. N. Alon any Y. Azar, J. Csirik, L. Epstein, S. V. Sevastianov, A. P. A. Vestjens, and G. J. Woeginger. On-line and off-line approximation algorithms for vector covering problems. In *Proc. 4th European Symposium on Algorithms*, LNCS, pages 406–418. Springer, 1996. To appear in *Algorithmica*.
3. S. F. Assmann. *Problems in Discrete Applied Mathematics*. PhD thesis, MIT, Cambridge, MA, 1983.
4. S. F. Assmann, D. S. Johnson, D. J. Kleitman, and J. Y. T. Leung. On a dual version of the one-dimensional bin packing problem. *J. Algorithms*, 5:502–525, 1984.

5. Y. Azar and L. Epstein. On two dimensional packing. In *Proc. 5th Scand. Workshop Alg. Theor.*, LNCS, pages 321–332. Springer, 1996.
6. B. S. Baker. A new proof for the First-Fit decreasing bin-packing algorithm. *J. Algorithms*, 6:49–70, 1985.
7. B. S. Baker, D. J. Brown, and H. P. Katseff. A $5/4$ algorithm for two-dimensional packing. *J. Algorithms*, 2:348–368, 1981.
8. B. S. Baker, D. J. Brown, and H. P. Katseff. Lower bounds for two-dimensional packing algorithms. *Acta Informatica*, 8:207–225, 1982.
9. B. S. Baker and E. G. Coffman. A tight asymptotic bound for Next-Fit-Decreasing bin-packing. *SIAM J. Alg. Disc. Meth.*, 2:147–152, 1981.
10. B. S. Baker, E. G. Coffman, and R. L. Rivest. Orthogonal packings in two dimensions. *SIAM J. Comput.*, 9:846–855, 1980.
11. B. S. Baker and J. S. Schwartz. Shelf algorithms for two-dimensional packing problems. *SIAM J. Comput.*, 12:508–525, 1983.
12. J. J. Bartholdi, J. H. vande Vate, and J. Zhang. Expected performance of the shelf heuristic for two-dimensional packing. *Oper. Res. Lett.*, 8:11–16, 1989.
13. J. L. Bentley, D. S. Johnson, F. T. Leighton, C. C. McGeoch, and L. A. McGeoch. Some unexpected expected behavior results for bin packing. In *Proc. 16th Annual ACM Symp. Theory of Computing*, pages 279–288, 1984.
14. D. Blitz, A. Van Vliet, and G. J. Woeginger. Lower bounds on the asymptotic worst-case ratio of on-line bin packing algorithms. Unpublished manuscript, 1996.
15. D. J. Brown. A lower bound for on-line one-dimensional bin packing algorithms. Technical Report R-864, Coordinated Sci. Lab., Urbana, Illinois, 1979.
16. J. Brzustowski. Can you win at Tetris? Master's thesis, The University of British Columbia, 1992.
17. H. Buriel. How to lose at Tetris. Technical report, The Geometry Center, Minneapolis, MN, 1996.
18. B. Chandra. Does randomization help in on-line bin packing? *Inform. Process. Lett.*, 43:15–19, 1992.
19. E. E. Chang, W. Wang, and M. S. Kankanhalli. Multidimensional on-line bin packing: An algorithm and its average-case analysis. *Inform. Process. Lett.*, 48:121–125, 1993.
20. F. R. K. Chung, M. R. Garey, and D. S. Johnson. On packing two-dimensional bins. *SIAM J. Alg. Disc. Meth.*, 3:66–76, 1982.
21. E. G. Coffman. An introduction to combinatorial models of dynamic storage allocation. *SIAM Rev.*, 25:311–325, 1983.
22. E. G. Coffman, M. R. Garey, and D. S. Johnson. Dynamic bin packing. *SIAM J. Comput.*, 12:227–258, 1983.
23. E. G. Coffman, M. R. Garey, and D. S. Johnson. Approximation algorithms for bin packing: a survey. In D. Hochbaum, editor, *Approximation algorithms for NP-hard Problems*, pages 46–93. PWS Publishing Company, 1997.
24. E. G. Coffman, D. S. Johnson, P. W. Shor, and R. R. Weber. Bin packing with discrete item sizes, part ii: average case behaviour of First Fit. Unpublished manuscript, 1996.
25. E. G. Coffman and G. S. Lueker. *Probabilistic analysis of Packing and Partitioning Algorithms*. John Wiley, New York, 1991.
26. E. G. Coffman, G. S. Lueker, and A. H. G. Rinnooy Kan. Asymptotic methods in the probabilistic analysis of sequencing and packing heuristics. *Management Science*, 34:266–290, 1988.

27. E. G. Coffman and P. W. Shor. Packing in two dimensions: asymptotic average-case analysis of algorithms. *Algorithmica*, 9:253–277, 1993.
28. E. G. Coffman, K. So, M. Hofri, and A. C. C. Yao. A stochastic model of bin packing. *Information and Control*, 44:105–115, 1980.
29. D. Coppersmith and P. Raghavan. Multidimensional on-line bin packing: Algorithms and worst case analysis. *Oper. Res. Lett.*, 8:17–20, 1989.
30. J. Csirik. Bin-packing as a random walk: a note on Knoedels paper. *Oper. Res. Lett.*, 5:161–163., 1986.
31. J. Csirik. An on-line algorithm for variable-sized bin packing. *Acta Informatica*, 26:697–709, 1989.
32. J. Csirik. The parametric behaviour of the First Fit decreasing bin-packing algorithm. *J. Algorithms*, 15:1–28, 1993.
33. J. Csirik, J. B. G. Frenk, G. Galambos, and A. H. G. Rinnooy Kan. Probabilistic analysis of algorithms for dual bin packing problems. *J. Algorithms*, 12:189–203, 1991.
34. J. Csirik, J. B. G. Frenk, and M. Labbe. Two dimensional rectangle packing: on line methods and results. *Discr. Appl. Math.*, 45:197–204, 1993.
35. J. Csirik and G. Galambos. An $O(n)$ bin packing algorithm for uniformly distributed data. *Computing*, 36:313–319, 1986.
36. J. Csirik, G. Galambos, and G. Turán. A lower bound on on-line algorithms for decreasing lists. In *Proc. of EURO VI*, 1984.
37. J. Csirik and B. Imreh. On the worst-case performance of the NkF bin packing heuristic. *Acta Cybernetica*, 9:89–105, 1989.
38. J. Csirik and D. S. Johnson. Bounded space on-line bin packing: best is better than first. In *Proc. 2nd ACM-SIAM Symp. Discrete Algorithms*, pages 309–319, 1991.
39. J. Csirik and E. Máté. The probabilistic behaviour of the NFD bin-packing heuristic. *Acta Cybernetica*, 7:241–246, 1986.
40. J. Csirik and V. Totik. On-line algorithms for a dual version of bin packing. *Discr. Appl. Math.*, 21:163–167, 1988.
41. J. Csirik and A. Van Vliet. An on-line algorithm for multidimensional bin packing. *Oper. Res. Lett.*, 13:149–158, 1993.
42. J. Csirik and G. J. Woeginger. Shelf algorithms for on-line strip packing. *Inform. Process. Lett.*, 63:171–175, 1997.
43. G. N. Frederickson. Probabilistic analysis for simple one- and two-dimensional bin packing algorithms. *Inform. Process. Lett.*, 11:156–161, 1980.
44. D. K. Friesen and M. A. Langston. Variable sized bin packing. *SIAM J. Comput.*, 15:222–230, 1986.
45. G. Galambos. Parametric lower bounds for on-line bin packing. *SIAM J. Alg. Disc. Meth.*, 7:362–367, 1986.
46. G. Galambos. Notes on Lee's harmonic fit algorithm. *Annales Univ. Sci. Budapest., Sect. Comp.*, 9:121–126, 1988.
47. G. Galambos. A 1.6 lower bound for the two-dimensional on-line rectangle bin packing. *Acta Cybernetica*, 10:21–24, 1991.
48. G. Galambos and J. B. G. Frenk. A simple proof of Liang's lower bound for on-line bin packing and the extension to the parametric case. *Discr. Appl. Math.*, 41:173–178, 1993.
49. G. Galambos, H. Kellerer, and G. J. Woeginger. A lower bound for on-line vector packing algorithms. *Acta Cybernetica*, 11:23–34, 1994.

50. G. Galambos and A. Van Vliet. Lower bounds for 1-, 2-, and 3-dimensional on-line bin packing algorithms. *Computing*, 52:281–297, 1994.
51. G. Galambos and G. J. Woeginger. Repacking helps in bounded space on-line bin packing. *Computing*, 49:329–338, 1993.
52. G. Gambosi, A. Postiglione, and M. Talamo. New algorithms for on-line bin packing. In *Proc. 1st Italian Conference on Algorithms and Complexity*, 1990.
53. M. R. Garey, R. L. Graham, D. S. Johnson, and A. C. C. Yao. Resource constrained scheduling as generalized bin packing. *J. Comb. Th. Ser. A.*, 21:257–298, 1976.
54. M. R. Garey and D. S. Johnson. *Computers and Intractability (A Guide to the theory of NP-Completeness)*. W. H. Freeman and Company, San Francisco, 1979.
55. H. Groemer. Covering and packing properties of bounded sequences of convex sets. *Mathematika*, 29:18–31, 1982.
56. E. F. Grove. On-line binpacking with lookahead. In *Proc. 6th ACM-SIAM Symp. Discrete Algorithms*, pages 430–436, 1995.
57. U. Hoffman. A class of simple stochastic on-line bin packing algorithms. *Computing*, 29:227–239, 1982.
58. M. Hofri. Two-dimensional packing: Expected performance of simple level algorithms. *Information and Control*, 45:1–17, 1980.
59. M. Hofri. A probabilistic analysis of the Next-Fit bin packing algorithm. *J. Algorithms*, 5:547–556, 1984.
60. M. Hofri. *Probabilistic analysis of algorithms*. Springer, New York, 1987.
61. M. Hofri and S. Kahmi. A stochastic analysis of the NFD bin packing algorithm. *J. Algorithms*, 7:489–509, 1986.
62. Z. Ivkovic and E. Lloyd. Fully dynamic algorithms for bin packing: Being myopic helps. In *Proc. 1st European Symposium on Algorithms*, LNCS, pages 224–235. Springer, 1993.
63. Z. Ivkovic and E. Lloyd. A fundamental restriction on fully dynamic maintenance of bin packing. *Inform. Process. Lett.*, 59:229–232, 1996.
64. Z. Ivkovic and E. Lloyd. Partially dynamic bin packing can be solved within $1+\epsilon$ in (amortized) polylogarithmic time. *Inform. Process. Lett.*, 63:45–50, 1997.
65. J. Januszewski and M. Lassak. On-line covering by boxes and by convex bodies. *Bull. Pol. Acad. Math.*, 42:69–76, 1994.
66. J. Januszewski and M. Lassak. On-line covering the unit cube by cubes. *Discrete Comput. Geom.*, 12:433–438, 1994.
67. J. Januszewski and M. Lassak. Efficient on-line covering of large cubes by convex bodies of at most unit diameters. *Bull. Pol. Acad. Math.*, 43:305–315, 1995.
68. J. Januszewski and M. Lassak. On-line covering the unit square by squares and the three-dimensional unit cube by cubes. *Demonstratio Math.*, 28:143–149, 1995.
69. J. Januszewski and M. Lassak. On-line packing the unit cube by cubes. unpublished manuscript, University of Bydgoszcz, Poland, 1996.
70. J. Januszewski, M. Lassak, G. Rote, and G. J. Woeginger. On-line q-adic covering by the method of the n-th segment and its application to on-line covering by cubes. *Beitr. Algebra Geom.*, 37:94–100, 1996.
71. D. S. Johnson. *Near-optimal bin packing algorithms*. PhD thesis, MIT, Cambridge, MA, 1973.
72. D. S. Johnson. Fast algorithms for bin packing. *J. Comput. System Sci.*, 8:272–314, 1974.

73. D. S. Johnson, A. Demers, J. D. Ullman, M. R. Garey, and R. L. Graham. Worst-case performance bounds for simple one-dimensional packing algorithms. *SIAM J. Comput.*, 3:256–278, 1974.
74. N. Karmarkar. Probabilistic analysis of some bin packing algorithms. In *Proc. 23rd IEEE Symp. Found. Comp. Sci.*, pages 107–118, 1982.
75. R. M. Karp, M. Luby, and A. Marchetti-Spaccamela. Probabilistic analysis of multi-dimensional binpacking problems. In *Proc. 16th Annual ACM Symp. Theory of Computing*, pages 289–298, 1984.
76. N. G. Kinnersley and M. A. Langston. On-line variable-sized bin packing. *Discr. Appl. Math.*, 22:143–148, 1988.
77. W. Knoedel. A bin packing algorithm with complexity $o(n \log n)$ and performance 1 in the stochastic limit. In J. Gruska and M. Chytil, editors, *Proc. 10th Symp. Math. Found. Comp. Science*, LNCS, pages 369–378. Springer, 1981.
78. W. Knoedel. Ueber das mittlere Verhalten von on-line Packungsalgorithmen. *EIK*, 19:427–433, 1983.
79. L. T. Kou and G. Markowsky. Multidimensional bin packing algorithms. *IBM J. Research and Development*, 21:443–448, 1977.
80. W. Kuperberg. On-line covering a cube by a sequence of cubes. *Discrete Comput. Geom.*, 12:83–90, 1994.
81. M. Lassak. On-line covering a box by cubes. *Beitr. Algebra Geom.*, 36:1–7, 1995.
82. M. Lassak and J. Zhang. An on-line potato-sack theorem. *Discrete Comput. Geom.*, 6:1–7, 1991.
83. C. C. Lee and D. T. Lee. A simple on-line bin packing algorithm. *J. Assoc. Comput. Mach.*, 32:562–572, 1985.
84. C. C. Lee and D. T. Lee. Robust on-line bin packing algorithms. Technical Report 83-03-FC-02, Department of Electrical Engineering and Computer Science, Northwestern University, Evanston, IL, 1987.
85. F. T. Leighton and P. Shor. Tight bounds for minimax grid matching with applications to the average case analysis of algorithms. *Combinatorica*, 9:161–187, 1989.
86. K. Li and K. H. Cheng. Heuristic algorithms for on-line packing in three dimensions. *J. Algorithms*, 13:589–605, 1992.
87. F. M. Liang. A lower bound for on-line bin packing. *Inform. Process. Lett.*, 10:76–79, 1980.
88. G. S. Lueker. An average-case analysis of bin packing with uniformly distributed item sizes. Technical Report 181, Dept. Information and Computer Science, University of California at Irvine, 1982.
89. W. Mao. Best-k-Fit bin packing. *Computing*, 50:265–270, 1993.
90. W. Mao. Tight worst-case performance bounds for Next-k-Fit bin packing. *SIAM J. Comput.*, 22:46–56, 1993.
91. A. Meir and L. Moser. On packing of squares and cubes. *J. Combin. Theory*, 5:126–134, 1968.
92. H. L. Ong, M. J. Magazine, and T. S. Wee. Probabilistic analysis of bin packing heuristics. *Oper. Res.*, 5:983–998, 1984.
93. P. Ramanan. Average case analysis of the Smart Next Fit algorithm. *Inform. Process. Lett.*, 31:221–225, 1989.
94. P. Ramanan, D. J. Brown, C. C. Lee, and D. T. Lee. On-line bin packing in linear time. *J. Algorithms*, 10:305–326, 1989.
95. P. Ramanan and K. Tsuga. Average case analysis of the modified harmonic algorithm. *Algorithmica*, 4:519–533, 1989.

96. W. T. Rhee and M. Talagrand. The complete convergence of best fit decreasing. *SIAM J. Comput.*, 18:909–918, 1989.
97. W. T. Rhee and M. Talagrand. The complete convergence of first fit decreasing. *SIAM J. Comput.*, 18:919–938, 1989.
98. W. T. Rhee and M. Talagrand. Dual bin packing of items of random size. *Math. Progr.*, 58:229–242, 1993.
99. W. T. Rhee and M. Talagrand. On line bin packing with items of random size. *Math. Oper. Res.*, 18:438–445, 1993.
100. W. T. Rhee and M. Talagrand. On line bin packing with items of random sizes — ii. *SIAM J. Comput.*, 22:1252–1256, 1993.
101. M. B. Richey. Improved bounds for harmonic-based bin packing algorithms. *Discr. Appl. Math.*, 34:203–227, 1991.
102. S. D. Shapiro. Performance of heuristic bin packing algorithms with segments of random length. *Inf. and Cont.*, 35:146–148, 1977.
103. P. W. Shor. The average-case analysis of some on-line algorithms for bin packing. *Combinatorica*, 6:179–200, 1986.
104. P. W. Shor. How to pack better than Best-Fit: Tight bounds for average-case on-line bin packing. In *Proc. 32nd IEEE Symp. Found. Comp. Sci.*, pages 752–759, 1991.
105. D. D. K. D. B. Sleator. A 2.5 times optimal algorithm for packing in two dimensions. *Inform. Process. Lett.*, 10:37–40, 1976.
106. A. Van Vliet. An improved lower bound for on-line bin packing algorithms. *Inform. Process. Lett.*, 43:277–284, 1992.
107. A. Van Vliet. *Lower and upper bounds for on-line bin packing and scheduling heuristics*. PhD thesis, Erasmus University, Rotterdam, The Netherlands, 1995.
108. A. Van Vliet. On the asymptotic worst case behaviour of harmonic fit. *J. Algorithms*, 20:113–136, 1996.
109. G. J. Woeginger. Improved space for bounded-space on-line bin packing. *SIAM J. Discr. Math.*, 6:575–581, 1993.
110. G. J. Woeginger and G. Zhang. Optimal on-line algorithms for variable-sized bin covering. Technical Report Woe-022, TU Graz, Austria, 1998.
111. A. C. C. Yao. New algorithms for bin packing. *J. Assoc. Comput. Mach.*, 27:207–227, 1980.
112. G. Zhang. A tight worst-case performance bound for AFB-k. Technical Report 015, Institute of Applied Mathematics, Academia Sinica, Beijing, China, 1994.
113. G. Zhang. Worst-case analysis of the FFH algorithm for on-line variable-sized bin packing. *Computing*, 56:165–172, 1996.